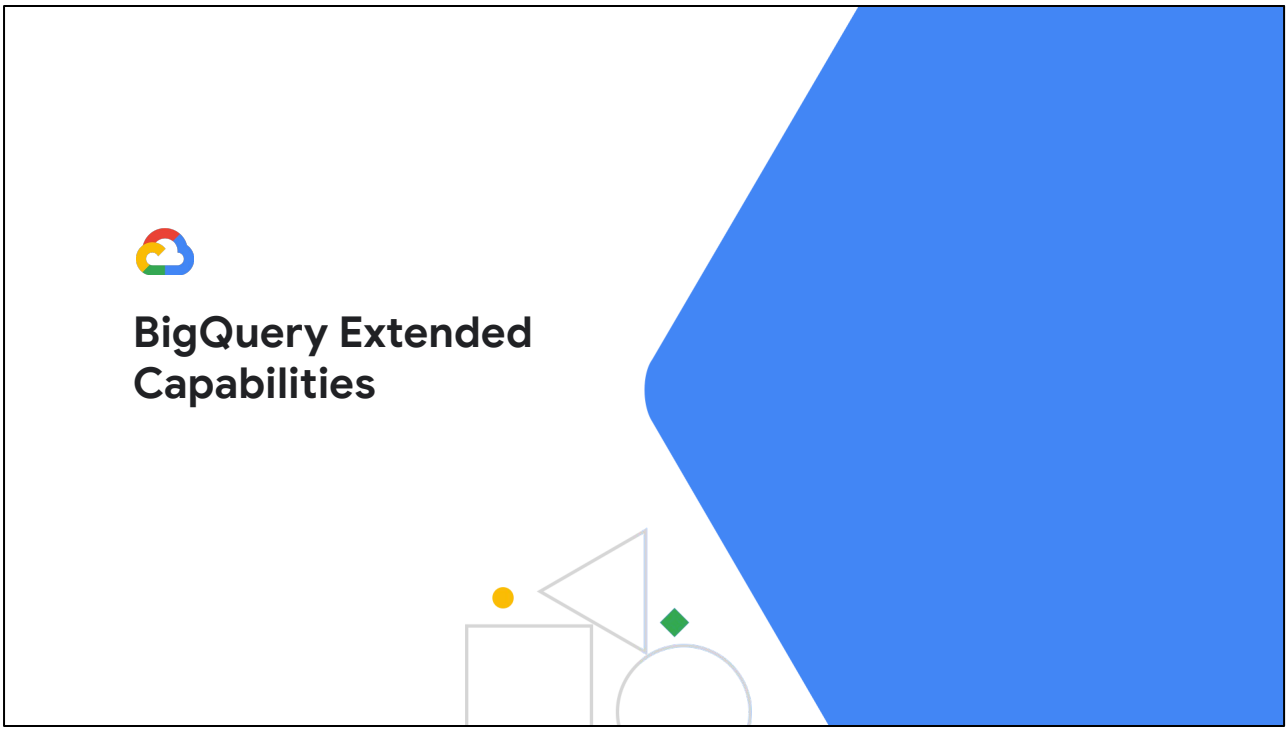


Welcome back. So if we're going to be asking customers to migrate to our data warehouse, to BigQuery, besides things like cost advantages we need to talk about things that they will be able to do that fundamentally they are not able to do on their existing data warehouse solution.

Proprietary + Confidential

Learn how to ...

Explore the Geographic Information Systems (GIS), GIS Visualization, and machine learning enhancements to BigQuery

- Describe the key features of BigQuery GIS and BigQuery ML
- Train and evaluate an ML model with BigQuery ML using the public NYC taxi cab dataset
- Use BigQuery ML to predict taxi fares using a BigQuery ML model
- Use BigQuery GIS functions to convert latitude and longitude data into geographical points
- Visualize BigQuery GIS data using BigQuery Geo Viz

Google Cloud

In this module, we'll explore the GIS (Geographic Information Systems) visualization and machine learning enhancements in BigQuery.

- We'll look at the key features of BigQuery GIS and BigQuery ML.
- We'll train and evaluate a machine learning model using BigQuery ML, using just plain old SQL, and using a massive dataset of New York taxi cab rates.
- And we'll use BigQuery ML to predict taxi fares using that model in a lab.
- Then we'll look at GIS functions to convert latitude and longitude data, polygon data, into geographies. We'll talk about what geographies are and why they're important.
- And finally, we'll look at how to visualize GIS data using BigQuery Geo Viz and use these in a lab as well.

Proprietary + Confidential

Agenda

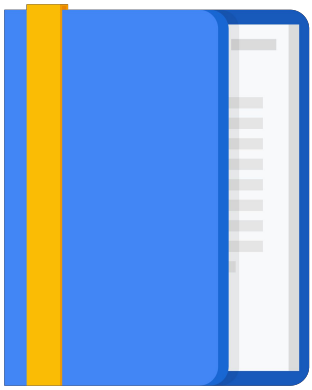
BigQuery GIS

Lab: Getting started with BigQuery GIS - New York City Bikes

BigQuery ML

Lab: Predict Taxi Fare with a BigQuery ML Forecasting Model

Summary

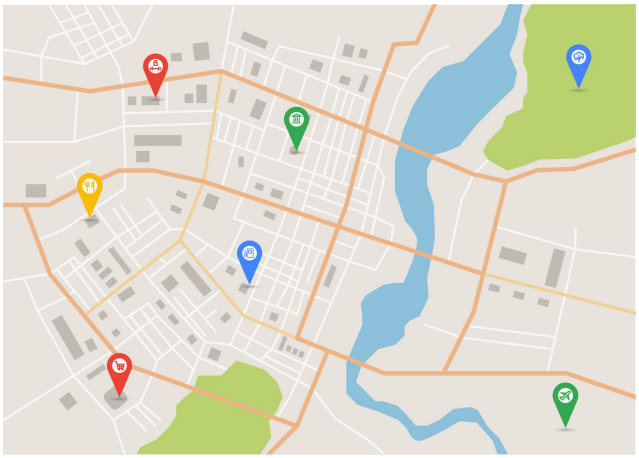


Google Cloud

Let's take a look at BigQuery GIS.

Proprietary + Confidential

Analyze geographic data with BigQuery GIS



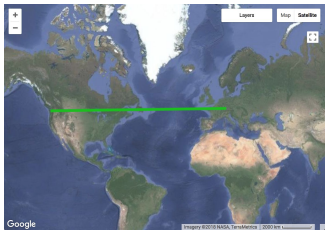
Google Cloud

With BigQuery GIS, you can analyze geographic data. Geographic data is data that you would normally see on maps, things like locations of cities, locations of your cars, locations of your assets. In a business situation, tracking where things are is extremely important. Knowing where your customers are relative to your store is an important question for a retail business to answer. But it's not just about points. It's also about things like polygons. For example, the shape of an urban area, the shape of a city, the shape of a state, the shape of a country. These are all very important. We want to figure out how many people live within a tax domain - the tax rate that somebody pays depends on which location they live in. And when we're going to be shipping items to that person, we need to know where they are. And that's an example of a geographic analysis that we need to be able to do. Other common analysis that people do is figuring out what the average income is of people within five kilometres of a particular location. These are the kinds of things that you want to do with geographic data where you're combining the geographic data, the locations of people, the locations of the polygons, with business data like the store location, the store sales, with how much a customer has spent in your site, etc.

Proprietary + Confidential

Geometry versus geography

Geometry



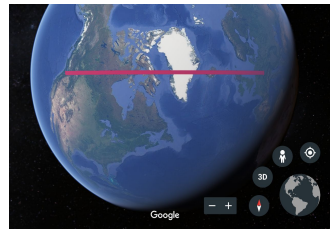
Shortest distance between 2 points is a straight line

Geography

Shortest distance is a great circle.

Wait, what??!!

Geography from space



Euclid was right

Google Cloud

Now we keep talking about geography. There are two terms here. Geometry versus geography. Geometry is where you're assuming that the earth is flat. The shortest distance between two points is a straight line.

In geography, the shortest distance between two points is a great circle.

And of course, this is correct, because when you look at the earth from space, a great circle looks like a line. So Euclid was right, but you need to use geographies not geometries.

Proprietary + Confidential

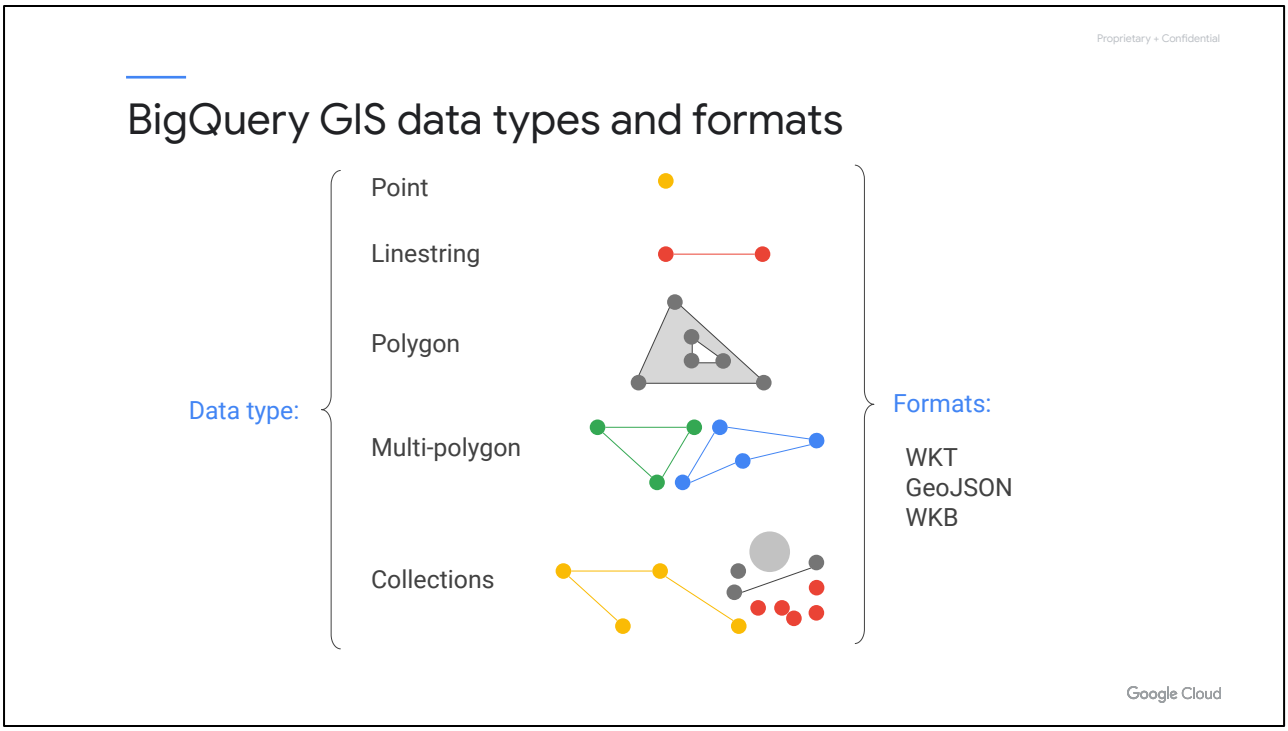
What is BigQuery GIS?

- Geospatial Information Systems
- BigQuery is the first MPP data warehouse to support geospatial data types and functions
- GEOGRAPHY data types represents a pointset on Earth
- Same S2 library as Google Maps
- BigQuery Geo Viz visualization tool

Google Cloud

So, what is BigQuery GIS?

- GIS stands for geographical or geospatial information systems.
- And BigQuery is the first massively parallel processing data warehouse to support geospatial data types, in particular geographies. And not just data types, but functions that operate on geographies that know that the earth is round.
- The geography data types represent a point set on the earth, and the data types are not just points, they include things like polygons. They include multi-polygons, polygons with holes in them, to be able to represent things like islands and island chains and so on. So the geography data types are pretty thorough and they cover a wide variety of different things that happen in geospatial analysis.
- These geography types are stored in a very efficient and effective way. It uses a library called S2 that was invented at Google and that's used in Google Maps as well.
- In addition, BigQuery GIS comes with a lightweight visualization tool called BigQuery Geo Viz that people look at.



So what are the data types and formats that BigQuery GIS supports?

The most basic data type is a **point**. So a latitude and longitude as an example is a point, although when you talk about geospatial analysis, you have a variety of different projections that are possible and points might be represented at other projections. But most commonly, business datasets get their locations from GPS and the GPS points are in something called a WGS ellipsoid, and it's in that ellipsoidal system that points are typically stored. So mostly you would use longitudes and latitudes that come from GPS, but BigQuery supports points and other protections as well.

The next thing that you might have is a **linestring** - the shortest distance between two points. As we said, even though we say it's a line, it could be a great circle if these points are far apart.

You could also have a **polygon** where you have multiple points, multiple linestrings, connected in a closed form. And as we said, a polygon could have holes in it, as shown in the diagram.

You might also have **multi-polygons**, an entity that consists of multiple polygons. Think of the country of Indonesia, for example. It's a bunch of different islands, each of which is a polygon, so that's multi-polygon.

You might also have **collections**. They have things that have some points, some

lines, some polygons in them. So this combination covers pretty much all of the geospatial data types that you will run into in a variety of different analysis,.

You have to get these data types into BigQuery, and the way you can ingest them could be as what is called WKT (well-known text), or WKB (well-known binary), or GeoJSON (geographic JSON).

BigQuery allows you to load geospatial data into a GEOGRAPHY column as long as the data is in one of these supported formats. There are other geospatial data formats like shapefiles, etc., or can convert them into these formats using open source geospatial tools. There's a lot of documentation that talks about how to convert these things into these geographic columns, and once you have them in a geographic column, you're ready to do geospatial analysis.

Proprietary + Confidential

Geospatial data formats

Well-known text (WKT)

Text format for describing individual geometry shapes using points, lines, polygons with optional holes, or a collection of points, lines, or polygons

```
POINT(-121 41)
```

Well-known binary (WKB)

Binary version of the WKT format

```
0101000000000000000000405ec000000
00000804440
```

GeoJSON

A more complex, JSON-based format for geometries and spatial features.

Used to describe:

- Individual geometry object
- Feature object
- Feature collection

```
{ "type": "Point", "coordinates": [-121,41] }
```

Google Cloud

Single points on earth can be described by just a (longitude, latitude) pair. For describing more complex geographies such as lines and polygons, BigQuery allows you to load geospatial data into a GEOGRAPHY column if the data is in a supported format.

WKT is a text format for describing individual geometry shapes using points, lines, polygons with optional holes, or a collection of points, lines, or polygons. To describe a spatial feature, WKT is usually embedded in some container file format, often CSV file, or in a database table. A file row or a table row usually corresponds to the spatial feature. The whole file or the table correspond to the feature collection.

WKB is binary version of the WKT format.

GeoJSON is a more complex, JSON-based format for geometries and spatial features. GeoJSON is used to describe one of the following:

- **An individual geometry object.** A single object can have a complex spatial shape, described as a union of points, lines, and polygons with optional holes. This is similar to WKT format usage.
- **A feature object.** A feature object is an object with a geometry, plus arbitrary additional named properties. This is similar to one row in a CSV file with a WKT column.
- **A featureCollection**. Feature collections are a set of feature objects similar to a table in a database or a CSV file with many rows and columns.

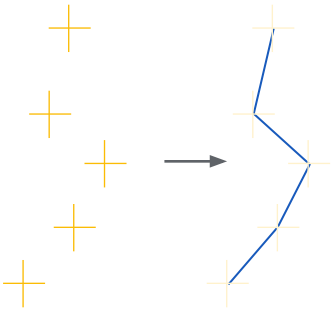
BigQuery GIS supports only individual geometry objects in GeoJSON. BigQuery GIS does not currently support GeoJSON feature objects, feature collections, or the GeoJSON file format.

Proprietary + Confidential

BigQuery GIS functions: Constructors

```
ST_GEOGPOINT(longitude, latitude)
ST_MAKELINE(geography_1, geography_2)
ST_MAKELINE(array_of_geography)
ST_MAKEPOLYGON(geography_expression)
ST_MAKEPOLYGON(geography_expression, array_of_geography)
ST_MAKEPOLYGONORIENTED(array_of_geography)
```

Build geographies from coordinates or existing geographies



Google Cloud

Constructors are functions that build new GEOGRAPHY values from coordinates or existing GEOGRAPHYs, such as ST_GEOGPOINT.

So once you get these points and polygons, what do you do with them? Well, you want to call functions on them. Just like you have a string, you can call uppercase, lowercase, etc. You can do CONCAT, you can do CONTAINS. What can you do with a polygon data type? What can you do with a point data type? Well, you have a wide variety of GIS functions that you could call.

Most basically, you can construct them. So this is how you construct a geographic point. You pass in a longitude, you pass in a latitude. If you have those two numbers, you can create a geographic point. And once you have those two points, you can make a line out of them by making an array of geographies and making a line. You can make a polygon out of them by having a bunch of different lines. You can also have an oriented polygon, and we'll talk about the difference between a polygon and an orientated polygon in a little bit. But the basic idea is, once you have your longitudes and latitudes, you can build a geography from it.

Proprietary + Confidential

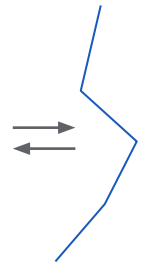
BigQuery GIS functions: Parsers and formatters

```
ST_GEOGFROMGEOJSON(geojson_string)
ST_GEOGFROMTEXT(wkt_string)
ST_GEOGFROMWKB(wkb_bytes)
```

```
ST_ASGEOJSON(geography_expression)
ST_ASTEXT(geography_expression)
ST_ASBINARY(geography_expression)
```

Create/export geographies
between formats

((0 0 0, 0 1 0, 1 1 0, 1 0 0, 0 0 0)), ((0 0 0, 0 1 0, 0 1 1, 0 0 1, 0 0 0)), ((0 0 0, 1 0 0, 1 0 1, 0 0 1, 0 0 0)),
((1 1 1, 1 0 1, 0 0 1, 0 1 1, 1 1 1)),
((1 1 1, 1 0 1, 1 0 0, 1 1 0, 1 1 1))



Google Cloud

You can also build the geography from parsing. **Parsers** are functions that create GEOGRAPHYs from an external format such as WKT and GeoJSON. For example,ST_GEOGFROMTEXT, creates a GEOGRAPHY from WKT. So you can create your geography from strings or bytes that represent that data.

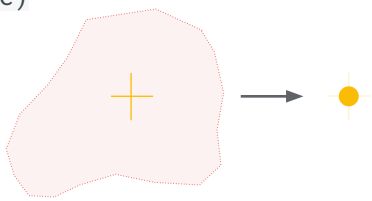
And you can also take the geography in BigQuery and export it out as a GeoJSON string or as a well-known text. So you can create and export geographies between different formats. **Formatters** are functions that export GEOGRAPHYs to an external format such as WKT and GeoJSON. For example, ST_ASTEXT formats a GEOGRAPHY to WKT.

Proprietary + Confidential

BigQuery GIS functions: Transformations

```
ST_INTERSECTION(geography_1, geography_2)
ST_UNION(geography_1, geography_2)
ST_UNION(array_of_geography)
ST_UNION_AGG(geography)
ST_DIFFERENCE(geography_1, geography_2)
ST_CENTROID(geography_expression)
ST_CLOSESTPOINT(geography_1, geography_2[,spheroid=FALSE])
ST_BOUNDARY(geography_expression)
ST_SNAPTOGRID(geography_expression, grid_size)
```

Create new geographies with similar properties



Google Cloud

Once you have a geography, what can you do with it? Very commonly, you want to find the intersection between two geographies. You want to establish if there's an overlap. For example, you could have one geography that represents the delivery area of your business, and another geography that represents where a particular neighborhood is. You want to establish if this neighborhood is within your delivery area.

Transformations are functions that generate a new GEOGRAPHY from others that adhere to some property. Examples include ST_INTERSECTION and ST_BOUNDARY.

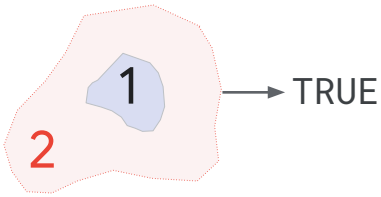
When you call ST_INTERSECTION, you may have two geographies that you want to combine. To do that, you do ST_UNION. You may have a geography and you want to find a center point. That's a CENTROID. You might have a zip code area. What is the closest point to that location? It may have a geography, like a river. What is the closest point on that river to a city? To my car? For this, you can do a CLOSESTPOINT. You can find the BOUNDARY of something. You can also do what's called a SNAPTOGRID, which takes a geography and rounds it off to a grid point. So you can call all of these transformations. And the reason we call them transformations is that the intersection of two geographies, the result is a geography. So the output of all of these functions is itself a geography. The output of the CENTROID function is a point, and a point is a geography.

Proprietary + Confidential

BigQuery GIS functions: Predicates

```
ST_CONTAINS(geography_1, geography_2)
ST_COVEREDBY(geography_1, geography_2)
ST_COVERS(geography_1, geography_2)
ST_DISJOINT(geography_1, geography_2)
ST_DWITHIN(geography_1, geography_2, distance[, spheroid=FALSE])
ST_EQUALS(geography_1, geography_2)
ST_INTERSECTS(geography_1, geography_2)
ST_INTERSECTSBOX(geography, lng1, lat1, lng2, lat2)
ST_TOUCHES(geography_1, geography_2)
ST_WITHIN(geography_1, geography_2)
```

Filter geographies (TRUE/FALSE)



Google Cloud

You can also do predicates. The predicate is something that returns a boolean value.

Predicates are functions that return TRUE or FALSE for some spatial relationship between two GEOGRAPHYs, or for some property of a geography. These functions are commonly used in filter clauses. For example, ST_DWITHIN is a predicate.

- Does this geography CONTAIN this other geography? Does this zip code CONTAIN the store?
- Are these two things DISJOINT?
- Is this vehicle within five kilometers of my store? You can call distance within (DWITHIN).
- You can determine if two things TOUCH.
- Is one geography WITHIN another geography or does this geography CONTAIN another geography. These two things can be very confusing. When you say ST_CONTAINS, you're saying: Does geography 1 contain geography 2? When you do ST_WITHIN, you're saying: Is geography 1 within geography 2. So in other words, they're the inverse of each other.

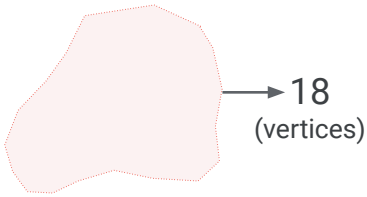
These all return, true or false, so you can use them in a WHERE clause.

Proprietary + Confidential

BigQuery GIS functions: Accessors

```
ST_ISEMPTY(geography_expression)
ST_ISCOLLECTION(geography_expression)
ST_DIMENSION(geography_expression)
ST_NUMPOINTS(geography_expression)
```

Access properties of geographies



Google Cloud

Other things that you can do is ask: Is this empty right now? Is it a 2D area, is it a 3D area? How many points are there?, etc. These are all ways to access properties of the geography.

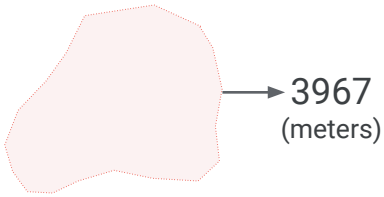
Accessors are functions that provide access to properties of a GEOGRAPHY without side-effects, such as ST_NUMPOINTS.

Proprietary + Confidential

BigQuery GIS functions: Measures

```
ST_DISTANCE(geography_1, geography_2[, spheroid=FALSE])
ST_LENGTH(geography_expression[, spheroid=FALSE])
ST_PERIMETER(geography_expression[, spheroid=FALSE])
ST_AREA(geography_expression[, spheroid=FALSE])
ST_MAXDISTANCE(geography_1, geography_2[, spheroid=FALSE])
```

Compute measurements of geographies



Google Cloud

You can also do measures. **Measures** are functions that compute measurements of one or more GEOGRAPHY, for example ST_DISTANCE.

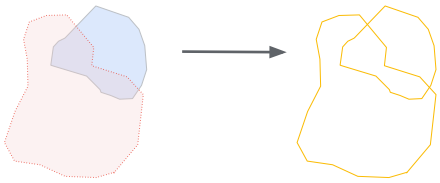
What is the DISTANCE between these two things? What's the LENGTH of this thing? What is the AREA of this thing? What is the MAXIMUM DISTANCE between these two things? All these calculations are carried out in a geography system. It understands that the earth is spheroidal - it’s not a perfect globe, it’s slightly flattened. It knows these things and computes knowing all of the aspects of the spheroidal representation of the earth.

Proprietary + Confidential

BigQuery GIS functions: Joins

```
JOIN on ST_INTERSECTS
JOIN on ST_DWITHIN
JOIN on ST_CONTAINS
JOIN on ST_WITHIN
JOIN on ST_COVERS
JOIN on ST_COVEREDBY
JOIN on ST_EQUALS
```

JOIN on geography Predicates statements



Google Cloud

You can do a WHERE clause, but you can also JOIN. You may have two tables, one table containing a geometry that's of store locations and another table containing a geometry of zip codes. You can join the two tables on the geography. This is very, very, very powerful. So you can use all these geography functions as a WHERE clause and as a JOIN clause as well.

Proprietary + Confidential

BigQuery GIS assumes that WKT points are defined in WGS-84

For importing geographies with spherical edges, use WKT

```
SELECT
  *,
  ST_GeogFromText(wkt) AS g
FROM
  mytable
```

For importing geographies with planar edges, often called "geometries", use GeoJSON

```
SELECT
  *,
  ST_GeogFromGeoJSON(geocol) AS g
FROM
  mytable
```

You may also exclude the original GeoJSON column from the results

```
SELECT
  * EXCEPT(geocol),
  ST_GeogFromGeoJSON(geocol) AS geocol
FROM
  mytable
```

Google Cloud

In BigQuery GIS, WKT points are positions on the surface of a WGS84 spheroid, expressed as longitude and geodetic latitude. An edge is a spherical geodesic between two endpoints. In GeoJSON, the coordinate system is explicitly WGS84 coordinates with planar edges.

To convert between these two kinds of edges, BigQuery GIS adds additional points to the line where necessary so the converted sequence of edges remains within 10 meters of the original line. This is a process known as tessellation or non-uniform densification. Currently, you cannot directly control the tessellation process.

Be sure to use the proper format. Most systems will either advertise their support for parsing geography, as opposed to geometry, from WKT, or they assume planar edges, in which case GeoJSON should be used as an interchange format.

Your coordinates should be longitude first, latitude second. If the geography has any long segments or edges then they must be tessellated, because BigQuery GIS interprets them as spherical geodesics, which may not correspond to the coordinate system where your data originated.

The World Geodetic System (WGS84) is the baseline reference coordinate system used by the Global Positioning System. In terms of absolute accuracy the uncertainty, or error, of WGS84 up to approximately 2 centimeters to the center mass (of the earth).

Proprietary + Confidential

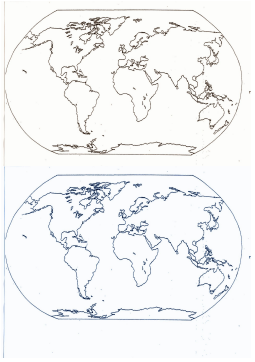
On a sphere, every polygon has a complimentary polygon described by the same boundary rings

The `ST_GeogFromText` function has an `oriented` flag:

1. (Default) Interpret the input as the polygon with the **smaller area**. Don't assume oriented polygons (`oriented = FALSE`).
2. Assume oriented polygons to allow loading polygons with an area larger than a hemisphere (`oriented = TRUE`). In this case, the points should listed so that the **area to the left** is the interior of the polygon.

When loading data from a WKT file, oriented=TRUE is the default, so be careful about point order!

GeoJSON doesn't have this ambiguity problem and should be preferred over WKT.



Google Cloud

On a sphere, every polygon has a complimentary polygon. For example, a polygon that describes the Earth's continents would have a complimentary polygon that describes the Earth's oceans. Because the two polygons are described by the same boundary rings, rules are required to resolve the ambiguity around which of the two polygons is described by a given WKT string.

When you load WKT and WKB strings from files or by using streaming ingestion, BigQuery GIS assumes the polygons in the input are oriented as follows: if you traverse the boundary of the polygon in the order of the input vertices, the interior of the polygon is on the left. BigQuery GIS uses the same rule when exporting geography objects to WKT and WKB strings.

When you build geography objects from a WKT string by using the ST_GeogFromText function, there are two options to determine the polygon described by the WKT string:

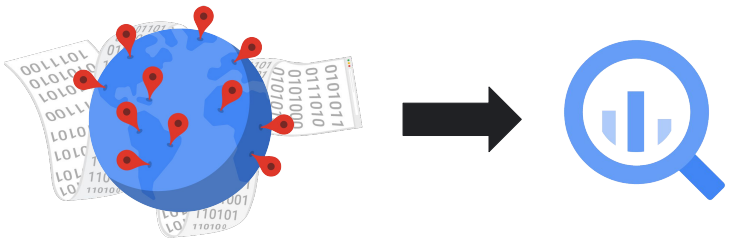
1. (Default) Interpret the input as the polygon with the smaller area. Do not assume oriented polygons (oriented = FALSE).
2. Assume oriented polygons to allow loading polygons with an area larger than a hemisphere (oriented = TRUE).

These rules do not apply when you load GeoJSON strings. Because GeoJSON strings are defined on a planar map, the orientation can be determined without ambiguity even if the input does not follow the orientation rule defined in GeoJSON

RFC 7946, section 3.1.6. — Polygon: counterclockwise external rings, clockwise internal rings.

Proprietary + Confidential

Loading BigQuery GIS data



```
ST_GEOGFROMGEOJSON(geojson_string)
ST_GEOGFROMTEXT(wkt_string)
ST_GEOGFROMWKB(wkb_bytes)
```

Google Cloud

When you load BigQuery GIS data into BigQuery, you can specify a GEOGRAPHY column in the table's schema definition. When you specify the column's data type as GEOGRAPHY, BigQuery GIS can detect whether the data is in WKT or GeoJSON format.

When you load GeoJSON geometry objects into a GEOGRAPHY column, they should be formatted as text strings, not as JSON objects. This is true even if the object is being loaded from a newline-delimited JSON file.

If you load data using schema auto-detect, geography values are loaded as STRINGs. Currently, schema auto-detect cannot detect geography columns.

Proprietary + Confidential

Transforming BigQuery GIS data

```
SELECT
  *,
  ST_GeogPoint(longitude, latitude) AS g
FROM
  mytable
```

Google Cloud

If your table contains separate columns for longitude and latitude, you can transform the values into geographies using standard SQL geography functions such as ST_GeogPoint. For example, if you have two double columns for longitude and latitude, you can create a geography column using the example query on screen.

BigQuery currently supports converting WKT and GeoJSON strings to geography types. Shapefiles and many other formats should be converted using external tools.

Proprietary + Confidential

Dealing with improperly formatted spatial data

```
SELECT
  geojson AS bad_geojson
FROM
  mytable
WHERE
  geojson IS NOT NULL
  AND SAFE.ST_GeogFromGeoJson(geojson) IS NULL
```

Google Cloud

When you load data into BigQuery, you may encounter invalid WKT or GeoJSON data from other tools that fails to be converted to a GEOGRAPHY column. For example, an error such as **Edge K has duplicate vertex with Edge N** indicates that the polygon has duplicate vertices (besides the first and last).

To avoid formatting issues, you can use a function that generates standards-compliant output. For example, when you export data from PostGIS, you can use the **ST_MakeValid** function to standardize the output.

To find or to ignore the improperly formatted data, use the SAFE function prefix to output the problematic data. For example, this query uses the SAFE prefix to retrieve improperly formatted spatial data.

Proprietary + Confidential

Spatial JOINs are joins of two tables with a predicate geographic function in the WHERE clause

```
#standardSQL
-- how many stations within 1 mile range of each zip code?
SELECT
    zip_code AS zip,
    ANY_VALUE(zcta_geom) AS polygon,
    COUNT(*) AS bike_stations
FROM
    `bigquery-public-data.new_york.citibike_stations` AS bike_stations,
    `bigquery-public-data.geo_us_boundaries.us_zip_codes` AS zip_codes
WHERE ST_DWithin(
         zip_codes.zcta_geom,
         ST_GeogPoint(bike_stations.longitude, bike_stations.latitude),
         1609.34)
GROUP BY zip
ORDER BY bike_stations DESC
```

Google Cloud

As we said, one of the big advantages is that we can do a JOIN using spatial data. Here's an example of a query - let's look at the FROM clause. There are two tables that we're reading from. One table consists of all of the bicycle stations and another table consists of the boundaries of US zip codes. We are reading from one table that says this is where all my bicycle stations are, and another table that says this is where all my zip codes are, this is the boundary of my zip codes. Looking at the WHERE clause, we're saying: Find me all the bike stations within 1,609 meters of the zip code at ST_GeogPoint, bike stations at longitude, bike stations at latitude. This is going to find everything inside the zip code and we're also going to add a 1.6 kilometer buffer, a 1 mile buffer around the zip code. In other words, we're finding all of the bicycle stations inside the zip code are within a mile of the zip code. That's a WHERE clause.

We're basically doing a typical query. Select the zip code, select the geometry, and select the number of bike stations. What we'll get back is the zip code, we'll get back the boundary of the zip code, and we'll get the number of bicycle stations in the zip code and within one mile of the zip code. All that we had to do was ST_DWithin (distance within), super powerful.

If you look at this query, we constructed the geographic point using ST_GeogPoint. This is not very efficient because it has to take longitude and latitude and find the S2 cell for it. It's okay if it's a few stations, but if you have a table that is hundreds of millions of rows, this can become computationally intensive.

Proprietary + Confidential

It's more performant to store the geography values in a BigQuery table

```
CREATE OR REPLACE TABLE
SELECT
  *,
  ST_GeogPoint(pLongitude, pLatitude) AS p
FROM
  mytable
```

Google Cloud

What you want to do is store the geography type itself in the table rather than store the longitudes and latitudes. And that's what we're doing here. We're constructing the geographic point and storing it in the table using CREATE OR REPLACE TABLE. Best practice is to make sure that your data type for geographic types is a geography, and it is not just latitudes and longitudes because it's going to be inefficient when you query it.

Proprietary + Confidential

BigQuery uses standard SQL predicate functions

- `ST_DWithin`
- `ST_Intersects`
- `ST_Contains`
- `ST_Within`
- `ST_Covers`
- `ST_CoveredBy`
- `ST_Equals`
- `ST_Touches`

Google Cloud

BigQuery implements optimized spatial JOINs for INNER JOIN and CROSS JOIN operators with standard SQL predicate functions.

Spatial joins aren't optimized:

- For LEFT, RIGHT or FULL OUTER joins.
- In cases involving ANTI joins.
- When the spatial predicate is negated.

A JOIN that uses the `ST_DWithin` predicate is optimized only when the distance parameter is a constant expression.

Proprietary + Confidential

To export data in GeoJSON format, use the ST_AsGeoJSON function

```
SELECT
  ST_AsGeoJSON(ST_MakeLine(ST_GeogPoint(1,1), ST_GeogPoint(3,2)))
```

```
{ "type": "LineString", "coordinates": [ [1, 1], [1.99977145571783,
1.50022838764041], [2.49981908082299, 1.75018082434274], [3, 2] ] }
```

Google Cloud

When you export spatial data from BigQuery, GEOGRAPHY column values are always formatted as WKT strings. To export data in GeoJSON format, use the ST_AsGeoJSON function.

If the tools you're using to analyze the exported data do not understand the GEOGRAPHY data type, you can convert the column values to strings using a geographic function such as **ST_AsText** or **ST_AsGeoJSON**. BigQuery GIS adds additional points to the line where necessary, so that the converted sequence of edges remains within 10 meters of the original geodesic line.

In this example, the query uses **ST_AsGeoJSON** to convert GeoJSON values to strings. The resulting data follows below.

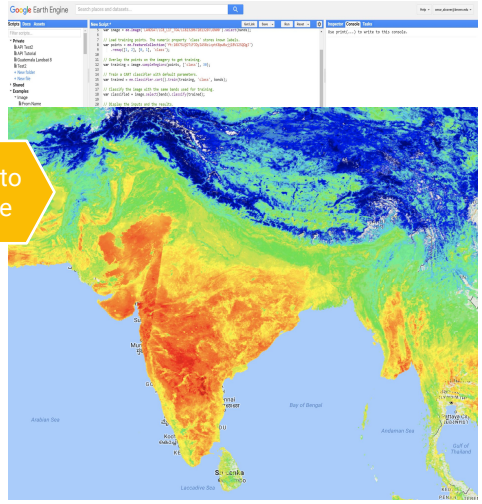
The GeoJSON line has two additional points. BigQuery GIS adds these points so that the GeoJSON line closely follows the same path on the ground as the original line.

Proprietary + Confidential

Visualize your findings in Google Earth Engine

Analyze in BigQuery → Export data to Cloud Storage → Load data into Earth Engine

CSV	EE UI
GeoJSON WKT	API
KML	CLI

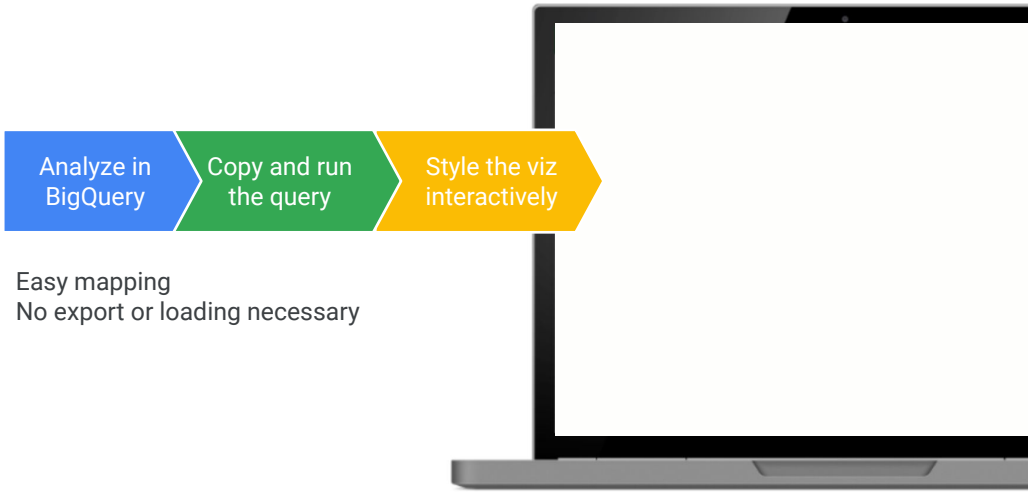


Google Cloud

Once you have these, you can also visualize your findings in any geospatial tool, including Google Earth Engine. You will export your data into things like GeoJSON that all these tools can read.

Proprietary + Confidential

Visualize your SQL query in BigQuery Geo Viz



Easy mapping
No export or loading necessary

Google Cloud

That's cool, but you also have this lightweight tool that comes in BigQuery, called BigQuery Geo Viz. You don't have to do exporting, you don't have to do loading. You can just put in your query, type it into the UI and click run to get all of your outputs. Not only do you get your outputs, you get the map associated with those outputs. What you're seeing is that the output is getting styled, you're seeing different colors and the colors are based on the particular data type.

Proprietary + Confidential

Visualize your findings

Load data in BigQuery → Analyze and style in Looker

Business users can perform GIS analysis without needing to understand GIS syntax

Combine BigQuery GIS and BigQuery ML

Google Cloud

You can also visualize your data with partner tools. In particular, Looker supports the visualization of BigQuery GIS data.

Proprietary + Confidential

Google Cloud Public Datasets go GEO

US_Boundaries

- Congressional districts
- Coastline
- Counties
- CONUS borders
- Combined Statistical Areas (CSAs)
- Metropolitan Statistical Areas (MSAs)
- States
- Urban areas
- Zip codes
- NOAA NWS forecast zones

Non-converted highlights

- Historic weather from NOAA
- 311 request data from SF, NYC and Austin
- Bike trips from NYC and London
- Natural Disasters (Earthquakes, tsunamis, hurricanes)
- Satellite imagery index from Landsat and more

Others

- NYC taxi trips (taxi zones)
- SF bike trips (station locations)
- NOAA lightning strikes
- Severe storm reports (hail, tornado, etc. 1950-2019)
- Geolite IP anonymization
- Solar energy potential

Google Cloud

One of the cool things to remember is that there's a lot of geographic data already in BigQuery.
My example query used U.S. zip codes, and those U.S. zip codes came from a public dataset program. Public datasets are free to use and they're all available in BigQuery. We have a variety of U.S. boundaries, congressional districts, coastlines, counties, statistical areas, urban areas, etc.

But it's not just the U.S. We also have data from a variety of other areas as well. You have EU boundaries, you have geo location and their energy potential, where cyclonic storms have been, and so on. There’s a whole bunch of geographic datasets available. There's also more and more geographic datasets coming into BigQuery, so look at our public datasets to see if the data that you need is there. If it's not there, reach out to us and we can look at whether we can get that and make it available to you.

Proprietary + Confidential

Agenda

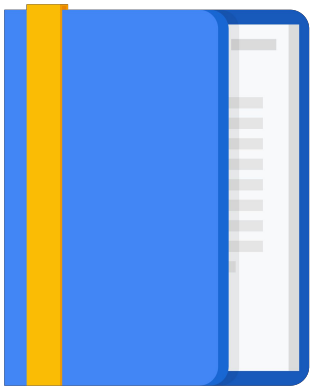
BigQuery GIS

Lab: Getting started with BigQuery GIS - New York City Bikes

BigQuery ML

Lab: Predict Taxi Fare with a BigQuery ML Forecasting Model

Summary



Google Cloud

You will now get the opportunity to use BigQuery GIS to easily analyze and visualize geospatial data in BigQuery.

Lab Intro

Getting started with BigQuery GIS
- New York City Bikes

In this lab, you will use BigQuery GIS to run a query that finds all the Citi Bike stations with more than 30 bikes available for rental.

Proprietary + Confidential

Lab objectives

- Use a BigQuery GIS function to convert latitude and longitude columns into geographical points
- Run a query that finds stations within 5 km of Times Square that have more than 30 bikes available for rental
- Visualize your results in BigQuery Geo Viz

Google Cloud

During the lab, you will learn how to:

- Use a BigQuery GIS function to convert latitude and longitude columns into geographical points,
- Run a query that finds stations within 5 km of Times Square that have more than 30 bikes available for rental, and
- Visualize your results in BigQuery Geo Viz.

Proprietary + Confidential

Agenda

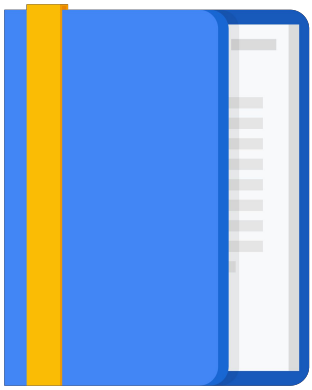
BigQuery GIS

Lab: Getting started with BigQuery GIS - New York City Bikes

BigQuery ML

Lab: Predict Taxi Fare with a BigQuery ML Forecasting Model

Summary



Google Cloud

We've looked at BigQuery GIS. Let's look at another amazing capability that's in BigQuery that isn't available in most other data warehouses - BigQuery machine learning.

Proprietary + Confidential

BigQuery ML has advantages over other approaches to using ML with a cloud-based data warehouse

- Build and run models using existing business intelligence tools and spreadsheets
- Train and access models using SQL, a language data analysts know
- There's no need to export data from the data warehouse

Google Cloud

Machine learning used to require extensive knowledge of machine learning frameworks. You needed to know Python, you needed to know queue runners, you needed to know how to read data in a distributed way. You needed to do a lot of things and you needed to know quite a bit of math and how to setup your matrix multiplications, and so on. It used to be hard. But in BigQuery you can do machine learning using just SQL and that has tremendous advantages.

- First of all, your data is in a data warehouse. It would be amazing if you could build and run models without ever moving the data out. Also, if you could use existing business intelligence tools, existing spreadsheets, and run machine learning models on them.
- You can create and access models using SQL, which is a language that data analysts know. One of the hardest problems in machine learning is understanding the data, and data analysts already understand the data. It would be great if we could allow them to build the models themselves. In order to do that, we want to make it available so that they can do this modeling in SQL, a language that they know.
- Importantly, there is no need to export the data from the data warehouse. This has tremendous benefits because there's a lot of security implications. If your data remains in BigQuery, remains in that secure environment, it makes life a lot easier, both from a contracts perspective and from a security perspective. There's a lot of advantages to being able to do machine learning without moving your data out.

Proprietary + Confidential

BigQuery ML allows you to build an ML model from a simple SQL syntax

- Leverage BigQuery processing power to build a model
- Auto-tuned learning rate
- Auto-split of data into training and test
- Null imputation
- Standardization of numeric features
- One-hot encoding of strings
- Class imbalance handling

Google Cloud

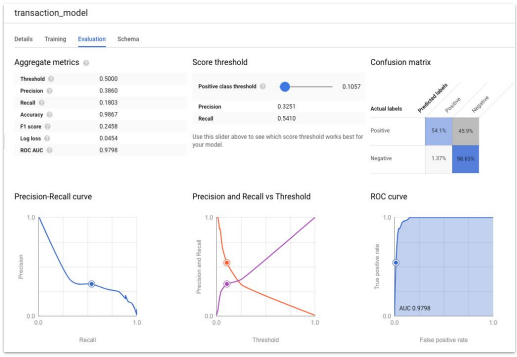
What we're talking about is building a machine learning model using simple SQL syntax.

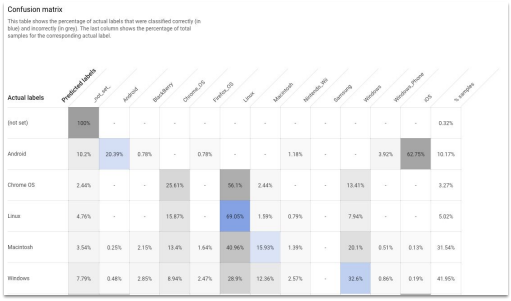
- We're going to use BigQuery's processing power to build a model. In fact, in our lab we'll do this model on billions of taxicab rides and it's not going to have a problem.
- Because we're using SQL, it's essentially aimed towards the data analysts. We're trying to make things as simple as possible and doing auto tuning of things like learning rate. You can set the learning rate, but if you don't set the learning rate, we will actually do a level of experimentation to figure out what learning rate works best on the data, and then turn around and apply that learning rate to the entire dataset.
- We will automatically split the data into training and test. You can control the way you want to split it, but if you don't specify anything, we make a default choice and assumption here.
- We automatically look at all of the data that's missing, that is null, and we figure out what value to replace it with. We do a null imputation.
- We take all your numeric data and we automatically scale them to be zero mean unit variance. You can see a pattern emerging here. The pattern is that we make reasonable defaults so that your SQL query can be extremely simple and still get great results.
- We take your string data and automatically one-hot encode it.
- If your label is imbalanced, we provide you a way to handle that class imbalance. This is important if you're trying to do things like fraud detection

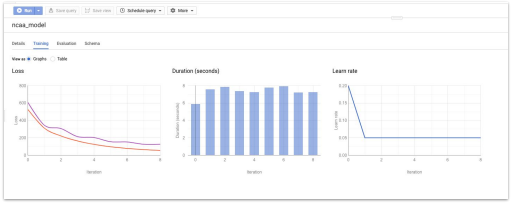
- because the chance of fraud is very small. The number of fraudulent transaction examples tend to be much smaller than the number of examples that you will have of good transactions.

Proprietary + Confidential

View model visualizations inside the BigQuery UI







Google Cloud

Once you've trained your model, you can go to the BigQuery user interface and you can look at the Evaluation tab. It will show you all of the graphs that data scientists typically generate. It will show you a confusion matrix, it will show you a ROC curve, it will show you the distribution of different columns. The ROC curve will even allow you to set the threshold and determine how a precision read changes, how a recall changes as you change the threshold. In other words, we're making BigQuery ML very easy to use, understanding that the end user is a data analyst, someone who knows the data, but is not necessarily a machine learning expert.

Proprietary + Confidential

Accessing BigQuery ML

- BigQuery web UI
- bq command-line interface
- BigQuery REST API
- External tools

Google Cloud

How do you access BigQuery ML? The best way to do it is to use a web UI because once you create the model, you have access to all of the training statistics, all of the evaluation statistics, etc.

But you can automate it using the BigQuery command line, the bq CLI, just as with any other BigQuery query.

You can access it using the REST API.

You can also access it through Looker, for example, where you can launch off a BigQuery machine learning training job.

Proprietary + Confidential

Create a model

```
CREATE OR REPLACE MODEL advdata.txtclass
OPTIONS(model_type='logistic_reg', input_label_cols=['source'])
AS

WITH extracted AS (
...
)
, ds AS (
SELECT ARRAY_CONCAT(SPLIT(title, " "), ['NULL', 'NULL', 'NULL', 'NULL', 'NULL']) AS words,
source FROM extracted
WHERE (source = 'github' OR source = 'nytimes' OR source = 'techcrunch')
)

SELECT
source,
words[OFFSET(0)] AS word1,
words[OFFSET(1)] AS word2,
words[OFFSET(2)] AS word3,
words[OFFSET(3)] AS word4,
words[OFFSET(4)] AS word5
FROM ds
```

Query to extract training data

Google Cloud

So how do you launch off a BigQuery ML training job? First, you write a query to extract the data. That is all of the query at the bottom of the slide: *WITH extracted AS*, etc. But notice what it ends up with. It ends up with a SELECT statement. It says: SELECT source, word 1, word 2, word 3, word 4, word 5. That's the training data. So if this training data has six columns, ultimately that's what you want to end up with. You want to end up with training data, a select query, that has the data.

What data does it have? It needs to have the input to the model and the correct output of the model. Because its training data, you have the correct output, you know the data. So you have the inputs that you want to train the model with and the correct output for that set of inputs. So once you have that select query, let's go back to the first three lines of the slide. What you're saying is I want to create or replace a model. So this is just like create or replace a table. The name of the model is txtclass (text class), that's like a table name, and advdata (advance data) is the name of the dataset. So we're creating a model named txtclass in the dataset advdata and we're parsing in that SELECT statement.

Now, which of those things is the label? Which is the correct answer? Which is the inputs? The way we do that is with that OPTIONS string. OPTIONS, input label column is source. So source is my label column - the correct answer. And then we say what the model type is. In this case, the model type is logistic regression. Logistic regression is a type of classification model. So in this case, it turns out to be a multiclass classification problem because a source column can be GitHub, New York Times, or TechCrunch. So three possible values. And this model will learn from five

words whether this article appeared in GitHub, New York Times, or TechCrunch.

As long as you have a dataset consisting of words in the article and where they appear you can you can train this model by saying CREATE OR REPLACE MODEL. So you extract the training data and you add CREATE OR REPLACE MODEL specifying the label column, which column is the correct answer, and the type of model that you want to train. I'll talk about the different types that are available shortly.

Proprietary + Confidential

Evaluate the model

```
SELECT * FROM ML.EVALUATE(MODEL advdata.txtclass)
```

precision	recall	accuracy	f1_score	log_loss	roc_auc
0.783	0.783	0.79	0.783	0.858	0.918

BigQuery ML splits the training data and reports evaluation statistics on the held-out set.

Actual labels \ Predicted labels	github	nytimes	techcrunch	% samples
github	88.8%	5.29%	5.9%	37.83%
nytimes	6.34%	70.92%	22.74%	31.26%
techcrunch	5.54%	19.35%	75.11%	30.9%

Google Cloud

Once you've created the model, BigQuery's going to go out and train the model. Then you can go to the UI and it'll show you precision, recall, accuracy, etc. You can also look at what's called the confusion table. This one is showing how many times GitHub is correctly identified as GitHub and how many times New York Times correctly identified as New York Times - that's the stuff in the blue. And then you see how many times New York Times is wrongly identified as TechCrunch, for example - that's the stuff in the white.

Remember, I said that if you don't specify otherwise, BigQuery would automatically take your training data and separate it into two parts. It will train on one part and use the other part to show you all these statistics. So it's showing you these statistics on the part of the data they're just withheld.

Now you can do this by looking at the UI, but you can also run the query: SELECT * FROM ML.EVALUATE, parsing in the name of the model. The name of our model is advdata.txtclass because that's the CREATE OR REPLACE MODEL that we did. That's why when we do the EVALUATE, we parse in advdata.textclass as the model to evaluate.

Proprietary + Confidential

Predict using trained model

```
SELECT * FROM ML.PREDICT(MODEL advdata.txtclass,(
     SELECT 'government' AS word1, 'shutdown' AS word2, 'leaves' AS word3, 'workers' AS word4, 'reeling' AS word5
     UNION ALL SELECT 'unlikely', 'partnership', 'in', 'house', 'gives'
     UNION ALL SELECT 'fitbit', 's', 'fitness', 'tracker', 'is'
     UNION ALL SELECT 'downloading', 'the', 'android', 'studio', 'project'
))
```

Row	predicted_source	word1	word2	word3	word4	word5
1	nytimes	government	shutdown	leaves	workers	reeling
2	nytimes	unlikely	partnership	in	house	gives
3	techcrunch	fitbit	s	fitness	tracker	is
4	techcrunch	downloading	the	android	studio	project

"Batch prediction"

https://console.cloud.google.com/bigquery?sq=663413318684:4d854a43ae93416eaeb349e1fc4888cb

Google Cloud

So you've evaluated the model - now how do you use it? The whole point of machine learning is that you train on data for which you know the answers and then you provide the imports to a new piece of data and say: What's the correct answer here, model? And the model does a prediction and tells you what the correct answer is. Remember that when we trained the model, the CREATE OR REPLACE MODEL, we had the correct answer, which was the source. But we had five inputs: word 1 word, 2, word 3, word 4, and word 5. So when we do a prediction, we have to specify five inputs: word 1, word 2, word 3, word 4, and word 5.

So that's what the ML.PREDICT is doing. Using SELECT * ML.PREDICT, here's a model: *advdata.txtclass*, and then saying word 1 is 'government', word 2 is 'shutdown', word 3 is 'leaves', word 4 is 'workers', and word 5 is 'reeling'.

So those are the five inputs. What's the right answer? And when you run that SELECT * ML.PREDICT, the query runs and comes back and says the predicted source is New York Times. So similarly, for 'unlikely partnership in house gives' it also says it's the New York Times, and for 'fitbits fitness tracker is', it says TechCrunch.

The reason I have the last one in red is that you said 'downloading the android studio project'. The correct answer is GitHub, but the model predicted it was TechCrunch. But then again, when you go back and look at the ML.EVALUATE, the model said that it is accurate 79 percent of the time. So if we gave it four items, chances are it's going to be wrong once.

Proprietary + Confidential

A model in BigQuery ML represents what an ML system has learned from the training data

- Linear regression for forecasting
- Binary logistic regression for classification
- Multiclass logistic regression for classification
- Matrix factorization
- Deep Neural Networks using TensorFlow
- Import TensorFlow models for prediction
- Feature preprocessing functions
- K-means clustering for data segmentation

Google Cloud

So what kind of model types are supported in BigQuery ML?

Well, you can do linear regression. If you want to forecast a number, if you want to predict a number, do linear regression.

If you want to do classification, do logistic regression.

Logistic regression also does multiclass. So what I just showed you was a multiclass classification because you had three possible answers: New York Times, BitHub, and TechCrunch.

In addition, you also have matrix factorization. This is what people tend to use if you are doing recommendation systems.

- You can train deep neural nets using TensorFlow.
- You can import a TensorFlow model and use it for prediction.
- You can do feature preprocessing functions.
- You can do K-means clustering.

Proprietary + Confidential

1a. Linear classifier: Logistic regression

```
create or replace model models.will_buy_banana_example
options(model_type='logistic_reg', input_label_cols=['banana']) AS

with purchases_data AS (
select
  CAST(user_id AS string) customer_id,
  CAST(zip_code AS string) home_zipcode,
  ['dawn', 'morning', 'afternoon', 'night'][OFFSET(CAST (TRUNC(order_hour_of_day/6) AS
INT64))] AS time_of_day,
  (SELECT 24852 IN (SELECT product_id FROM UNNEST(order_lines))) AS banana
FROM operations.orders_with_lines
JOIN operations.customers_loyalty
ON user_id = id
)
select * from purchases_data
```

Google Cloud

We looked at logistic regression at a little earlier. Logistic regression is just another name for a linear classifier. It's a linear model that categorizes into different types. We looked at a multiclass earlier, so let's look at a binary - yes or no, true or false. This model is saying: Will this customer buy a banana or will they not buy a banana? If you want to train such a model, you have to go back and look at all the previous transactions and say: Did they buy a banana or did they not buy a banana? Again, the whole idea with machine learning is that you have historical data where you know the correct answer and you use that in the future. You provide the same inputs and the model tells you what the answer would be.

So in this case, we are training the model based on three inputs: the customer I.D, the home zip code and the time of the day. So three inputs and one output. The output is banana. That is set by the product_id banana in the order. We go back and look at the historical orders that have happened and ask: Is the banana in that list of orders? And if it is, then they bought a banana. For this grocery store, 24852 is the item code for a banana. If that item code is in the list of all the orders, they bought a banana.

Having done that, we now have three inputs. We say if they bought a banana or not, and that's the data, select * from purchases_data on which we do a create or replace model. The name of the model is will_buy_banana_example, and the name of the dataset is models. And the label column as you see as banana. So the other three columns are the input and it's a logistic regression. So this one is going to be True or False - they buy a banana or they don’t buy a banana. You train this model and then you can use this model to predict whether or not they buy a banana.

Proprietary + Confidential

1b. DNN classifier

```
create or replace model models.will_buy_banana_example
options(model_type='dnn_classifier', hidden_units=[64, 8], input_label_cols=['banana']) AS

with purchases_data AS (
select
  CAST(user_id AS string) customer_id,
  CAST(zip_code AS string) home_zipcode,
  ['dawn', 'morning', 'afternoon', 'night'][OFFSET(CAST (TRUNC(order_hour_of_day/6) AS
INT64))] AS time_of_day,
  (SELECT 24852 IN (SELECT product_id FROM UNNEST(order_lines))) AS banana
FROM operations.orders_with_lines
JOIN operations.customers_loyalty
ON user_id = id
)
select * from purchases_data
```

Google Cloud

You can take the same query and replace the model type with dnn_classifier. Now you're saying I want you to train a classifier to see whether they would buy a banana or not. That is going to be a deep neural net. It's going to have two layers, look at the hidden_units, the first is going to have 64 nodes and the second layer is going to have 8 nodes. The rest of the code is exactly the same. We are now doing deep learning in SQL, which is pretty cool.

Proprietary + Confidential

1c. xgboost classifier

```
create or replace model models.will_buy_banana_example
options(model_type='boosted_tree_classifier', max_tree_depth=5, input_label_cols=['banana'])
AS

with purchases_data AS (
select
  CAST(user_id AS string) customer_id,
  CAST(zip_code AS string) home_zipcode,
  ['dawn', 'morning', 'afternoon', 'night'][OFFSET(CAST (TRUNC(order_hour_of_day/6) AS
INT64))] AS time_of_day,
  (SELECT 24852 IN (SELECT product_id FROM UNNEST(order_lines))) AS banana
FROM operations.orders_with_lines
JOIN operations.customers_loyalty
ON user_id = id
)
select * from purchases_data
```

Google Cloud

You can also train an *xgboost classifier*, the decision tree classifier. In order to do that, the query is exactly the same. The model type becomes *boosted_tree_classifier*, and because it's now a boosted tree, we have to say how big the tree is, a *max_tree_depth* of 5. Then you train the model and that's pretty much it. What you get back is a boosted tree classifier.

Regardless of which you trained, the way you do the prediction will be exactly the same. The way you do the prediction will be just like SELECT * FROM ML.PREDICT, model my banana example, and parse in those three inputs that we trained on. So the model knows what type of model it is and it'll do the right thing as far as prediction goes. All we need to do is call SELECT * FROM ML.PREDICT.

Proprietary + Confidential

2a. Linear regression

```
CREATE OR REPLACE MODEL ch09edu.bicycle_model
OPTIONS(input_label_cols=['duration'],
        model_type='linear_reg')
AS

SELECT
  duration
  , start_station_name
  , CAST(EXTRACT(dayofweek from start_date) AS STRING)
         as dayofweek
  , CAST(EXTRACT(hour from start_date) AS STRING)
         as hourofday
FROM
  `bigquery-public-data.london_bicycles.cycle_hire`
```

Google Cloud

We looked at classification, but we also have the same models available for regression. The difference between classification and regression is that with classification you're trying to predict a category. True or false, which is binary classification, or New York Times, GitHub, TechCrunch, which has the names of classes. With regression, you're trying to predict a number, you're trying to predict a quantity.

In this case, we're looking at a dataset of bicycle rides. Historically, when somebody rents it out and returns it, we know how long they're rented the bicycle for. We know the duration of the bicycle ride. What we're doing now is trying to predict how long someone is going to keep a bicycle when they rent it. In order to do that, we pull out the data: duration, starting station name, day of the week, and hour of the day.

Now, which of these is the label? The duration - the thing that we're trying to predict. The other three are inputs: start station name, day of the week, and hour of the day. They're all inputs to the model and we use it to train the model. So you train this model, saying it's a linear regression because you want it to learn how to predict the duration. Now you have a model where if we give it a starting station name, Thursday, and three o'clock, it'll tell you: At this station, bicycles rented at 3 o'clock tend to be rented for 43 minutes. It gives you that prediction.

Proprietary + Confidential

2b. DNN regression

```
CREATE OR REPLACE MODEL ch09edu.bicycle_model
OPTIONS(input_label_cols=['duration'],
        model_type='dnn_regressor', hidden_units=[32, 4])
AS

SELECT
  duration
  , start_station_name
  , CAST(EXTRACT(dayofweek from start_date) AS STRING)
         as dayofweek
  , CAST(EXTRACT(hour from start_date) AS STRING)
         as hourofday
FROM
  `bigquery-public-data.london_bicycles.cycle_hire`
```

Google Cloud

Again, keep the query exactly the same, just change the model type from linear regression to a *dnn_regressor*. Now you're training a deep model. As a general rule of thumb, train a linear model first, get it working, and then try out the DNN. When you try out a DNN, you have to experiment with the number of nodes because you never know which one's going to work well. The numbers in this example, 32 and 4, are just arbitrary. We have to try out a variety of numbers to see what works well on this particular dataset. That is done purely by experimentation.

Proprietary + Confidential

2c. xgboost regression

```
CREATE OR REPLACE MODEL ch09edu.bicycle_model
OPTIONS(input_label_cols=['duration'],
        model_type='boosted_tree_regressor')
AS

SELECT
  duration
  , start_station_name
  , CAST(EXTRACT(dayofweek from start_date) AS STRING)
         as dayofweek
  , CAST(EXTRACT(hour from start_date) AS STRING)
         as hourofday
FROM
  `bigquery-public-data.london_bicycles.cycle_hire`
```

Google Cloud

You can also do *xgboost regressor*. In this case, I'm not specifying a *max_tree_depth*, so BigQuery applies appropriate defaults. For the most part, let BigQuery do its default thing.

Proprietary + Confidential

Train on TensorFlow, predict with BigQuery

```
CREATE OR REPLACE MODEL advdata.txtclass_tf2
OPTIONS (model_type='tensorflow',

model_path='gs://cloud-training-demos-ml/txtcls/trained_finetune_native/export/exporter/1549825580/*')
```

```
SELECT
  input,
  (SELECT AS STRUCT(p, ['github', 'nytimes', 'techcrunch'][ORDINAL(s)]) prediction FROM
    (SELECT p, ROW_NUMBER() OVER() AS s FROM
      (SELECT * FROM UNNEST(dense_1) AS p))
  ORDER BY p DESC LIMIT 1).*

FROM ML.PREDICT(MODEL advdata.txtclass_tf2,
(
SELECT 'Unlikely Partnership in House Gives Lawmakers Hope for Border Deal' AS input
UNION ALL SELECT "Fitbit\'s newest fitness tracker is just for employees and health insurance members"
UNION ALL SELECT "Show HN: Hello, a CLI tool for managing social media"
))
```

Google Cloud

Another cool thing is that you can train on Vertex AI here you get a TensorFlow model. You can load it into BigQuery and you can do ML.PREDICT with that model just as if it was a model that was trained in BigQuery itself. This is super powerful because now you can train a really complex natural language processing model on Vertex AI, bring the model into BigQuery, and then do an ML.PREDICT. The cool thing about ML.PREDICT is that now you're getting batch prediction directly in the data warehouse. All that you have to do is schedule this query once a day to say, OK, what are the numbers for tomorrow? You want to do a prediction? Just schedule a query with an ML.PREDICT, and that's pretty much it.

Proprietary + Confidential

Matrix factorization: Recommend products to users

```
create or replace model models.suggested_products_1or2_example
options(model_type='matrix_factorization',
        user_col='user_id', item_col='product_id', rating_col='rating',
        l2_reg=10)
AS

with purchases AS (
 select product_id, user_id from
 operations.orders_with_lines, unnest(order_lines)
),

total_purchases as (
select product_id, user_id, count(*) as numtimes
from purchases
group by product_id, user_id
)

select
product_id, user_id,
IF(numtimes < 2, 1, 2) AS rating
FROM total_purchases
```

Google Cloud

The other type of model that BigQuery supports is called matrix factorization. Matrix factorization is the simplest of the machine learning models that does collaborative filtering. It allows you to do product recommendations. When you want to do that, you need to have a table, and all it needs are three columns: a user_id, a product_id, and a rating. If you have historical data with those three columns, you're in business. You can train a recommendation model.

Let's look at the query here: *with purchases AS*. Notice that I'm pulling out a product_id and a user_id. But I don't have a rating because I'm a grocery store. I don't ask people to come rate my bananas, my apples, and my boxes of cereal. So what do I do? Well, I have a loyalty card because I'm a grocery chain. I know when people have come back and bought this item, so I can say that if somebody buys an item only once, and they never buy it again, then they don't like it. But if they buy the item more than twice, then they do like it. This is why, in the rating column at the very bottom of the slide, if the number of times a product is purchased is less than 2, the rating is 1. Otherwise, everything is 2. So a rating is made based on how many times a customer has bought this item.

But I'm in business. I have a product_id, I have a user_id, and I have a rating. Now look at the top of the slide. *Create or replace model* - the model type is matrix factorization. We say, what is the name of the column that represents the user? What is the name of the column that represents the item? And what is the name of the column that represents the rating, and we go.

Now one of the things with machine learning is that it's often good to experiment. If we look at the model's documentation, look at the documentation on the BigQuery site for *create a replace model*, you will find a lot of the options where you can try things out and experiment. One of these things is called L2 regularization, the way of preventing overfitting. It turns out that on this grocery dataset, because there are lots of items that a grocery chain carries and most people only buy the same items over and over again, there's a high chance of overfitting. One way to reduce overfitting is to use that option called L2 regularization. So that's why you see that option there. So I can go look at a model, look at what options exist. It tells you what the range of those numbers is, try them out.

Proprietary + Confidential

So what do we recommend for a given set of users?

```
with users AS (
SELECT
user_id, count(*) as num_orders
from operations.orders_with_lines
group by user_id
order by num_orders desc
limit 10
),

products as (
select product_id, count(*) as num_orders
from operations.orders_with_lines, unnest(order_lines)
group by product_id
order by num_orders desc
limit 10
)

SELECT * FROM ML.PREDICT(MODEL models.suggested_products_1or2,
(SELECT user_id, product_id
FROM users, products)
)
```

Row	predicted_rating	user_id	product_id
1	1.5746015507788755	101797	26209
2	1.8070705987455633	101797	13176
3	1.7171094544245578	101797	27845
4	1.9763373899260837	101797	47209
5	1.8659380090171271	101797	21137
6	1.721610848530093	101797	47766
7	1.9516130703939483	101797	21903

Google Cloud

So, we trained the model, but what do you do with it? Well, you want to call ML.PREDICT, always, always. You do *create or replace model*, train the model, and then use that model by calling ML.PREDICT. So how do you use a recommendation model? The way you use a recommendation model is you say: when a user comes in, should we recommend this item to them or not? You want to recommend items that they like, so you do a SELECT * FROM ML.PREDICT, parsing in the name of the model: *suggested_products_1or2_example*. So that's what this is, a lot of suggested products. Then we do SELECT * FROM ML.PREDICT and we pull out the two inputs: *user_id*, and *product_id*.

Why are we not providing the rating? Because a rating is a thing that we want to predict. Rating is the label. The two inputs are *user_id* and *product_id* and the thing we want to predict is the rating. So we're parsing the *user_id* and the *product_id* and the model tells us what it thinks the user would rate a given product.

What we're seeing in this table, is whether the user will buy the product twice or not because that's how we created the rating. Remember, if a product is bought less than twice the rating is 1, if bought more than twice, the rating is 2. From these results, we should be certainly be recommending the products in rows 4 and 7 and probably not those in rows 1 and 3.

Proprietary + Confidential

Clustering

```
CREATE OR REPLACE MODEL demos_eu.london_station_clusters
OPTIONS(model_type='kmeans', num_clusters=4, standardize_features = true) AS



WITH hs AS ...,
stationstats AS ...

SELECT * except(station_name, isweekday)
from stationstats
```

1. 4 clusters (hardcoded)
2. Standardize features since different dynamic ranges
3. Remove the cluster “id” fields (keep just the attributes)

Google Cloud

The other model that's also supported in BigQuery ML is called clustering. Unlike everything we've spoken about so far, with clustering you don't know the true answer. You just want to discover natural clusters in the data. In order to do that, you provide a bunch of information that you know about a particular thing, and say: Find clusters for me. The model type is K-means, and you specify how many clusters you want.

Proprietary + Confidential

Which cluster?

```
WITH hs AS ...,
stationstats AS ...,

SELECT * except(nearest_centroids_distance)
FROM ML.PREDICT(MODEL demos_eu.london_station_clusters,
(SELECT * FROM stationstats WHERE REGEXP_CONTAINS(station_name, 'Kennington')))
```

Row	CENTROID_ID	station_name	isweekday	duration	num_trips	bikes_count
1	3	Kennington Lane Tesco, Vauxhall	weekday	911.5810637908974	5471	9
2	3	Kennington Lane Rail Bridge, Vauxhall	weekday	979.3919952622995	20263	19
3	4	Doddington Grove, Kennington	weekday	1397.7189755200225	7067	28
4	4	Kennington Cross, Kennington	weekday	911.5238777770538	15349	35

Google Cloud

What comes back is clusters. Each of those clusters has a centroid, which is the center point of that cluster. When you do ML.PREDICT, it will tell you which cluster every item belongs in. On the previous slide, when we did clustering, we did the clustering of stations. What we're saying is that this particular station is in this particular cluster. For example, Kennington Cross, which is the fourth item, is in CENTROID_ID number 4. That’s what it’s telling us.

Proprietary + Confidential

Find cluster attributes

```
WITH T AS (
SELECT
centroid_id,
ARRAY_AGG(STRUCT(numerical_feature AS name, ROUND(feature_value,1) AS value) ORDER BY
centroid_id) AS cluster
FROM ML.CENTROIDS(MODEL demos_eu.london_station_clusters)
GROUP BY centroid_id
)
SELECT
CONCAT('Cluster#', CAST(centroid_id AS STRING)) AS centroid,
(SELECT value from unnest(cluster) WHERE name = 'duration') AS duration,
(SELECT value from unnest(cluster) WHERE name = 'num_trips') AS num_trips,
(SELECT value from unnest(cluster) WHERE name = 'bikes_count') AS bikes_count,
(SELECT value from unnest(cluster) WHERE name = 'distance_from_city_center') AS
distance_from_city_center
FROM T
ORDER BY centroid_id ASC
```

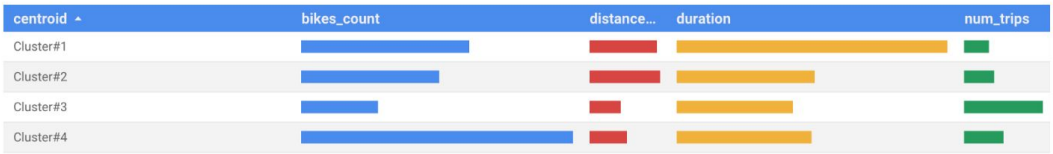
Google Cloud

But what does centroid_id number 4 mean? In order to do that, you do SELECT * FROM ML.CENTROIDS, and it tells you all of the information about the centroids.

Proprietary + Confidential

Visualize attributes in the Evaluation tab of the Google Cloud console

Row	centroid	duration	num_trips	bikes_count	distance_from_city_center
1	Cluster#1	3079.5	3026.1	14.0	6.2
2	Cluster#2	1564.0	3635.1	11.5	6.5
3	Cluster#3	1319.6	9654.8	6.4	2.9
4	Cluster#4	1527.7	4846.8	22.6	3.5

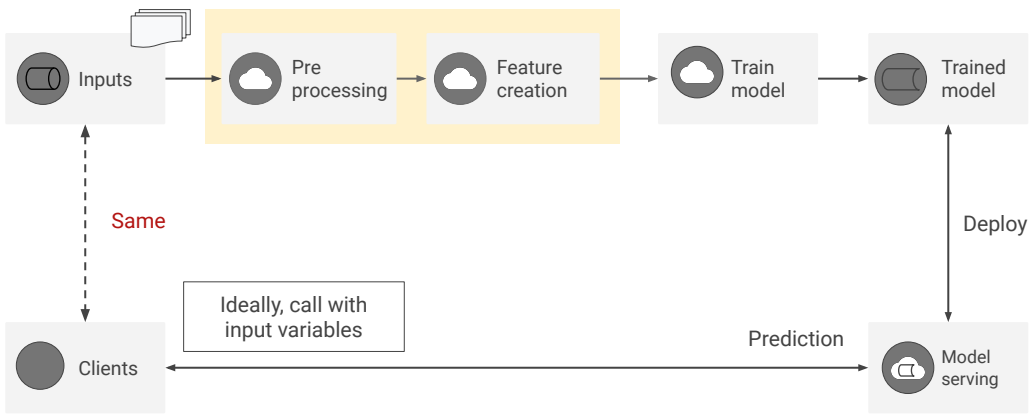


Google Cloud

These are all SQL queries, but a much simpler way is to go to the BigQuery console and go to the Evaluation tab. It displays all this for you. It shows you the clusters and cluster attributes, like the average value within those clusters. You will still do SELECT * FROM ML.PREDICT to figure out which particular cluster any particular item belongs to.

Proprietary + Confidential

Who does transformation during prediction?



Google Cloud

The other thing that BigQuery ML supports is this idea of transformation. This is a really, really hard problem in machine learning because normally you don't just take your data and use it as this to train a model. All the examples so far have been using raw data, but that's not normal. Normally, you would take your data and do some kind of feature creation from it. In fact, we actually did that once. When we did the bicycle model, we took the timestamp, the time which somebody rented the bicycle, and we extracted out the day of the week and the hour of the day from it. That's the kind of transformation that we need to do.

Proprietary + Confidential

TRANSFORM ensures transformations are automatically applied during ML.PREDICT

```
CREATE OR REPLACE MODEL ch09edu.bicycle_model
OPTIONS(input_label_cols=['duration'],
        model_type='linear_reg')
AS

SELECT
  duration
  , start_station_name
  , CAST(EXTRACT(dayofweek from start_date) AS STRING)
         as dayofweek
  , CAST(EXTRACT(hour from start_date) AS STRING)
         as hourofday
FROM
  `bigquery-public-data.london_bicycles.cycle_hire`
```

```
SELECT * FROM ML.PREDICT(MODEL ch09edu.bicycle_model,(
  350 AS duration
  , 'Kings Cross' AS start_station_name
  , '3' as dayofweek
  , '18' as hourofday
))
```

```
CREATE OR REPLACE MODEL ch09edu.bicycle_model
OPTIONS(input_label_cols=['duration'],
        model_type='linear_reg')
TRANSFORM(
  SELECT * EXCEPT(start_date)
  , CAST(EXTRACT(dayofweek from start_date) AS STRING)
         as dayofweek
  , CAST(EXTRACT(hour from start_date) AS STRING)
         as hourofday
)
AS
SELECT
  duration, start_station_name, start_date
FROM
  `bigquery-public-data.london_bicycles.cycle_hire`
```

```
SELECT * FROM ML.PREDICT(MODEL ch09edu.bicycle_model,(
  350 AS duration
  , 'Kings Cross' AS start_station_name
  , CURRENT_TIMESTAMP() as start_date
))
```

Google Cloud

When we trained this model, we trained it on the day of the week, hour of the day, and the station name. The station name is easy, people know what station names are. So when we do the PREDICT, we can say the station name is Kings Cross, it’s a real station name and understood. The problem is extracting the day of the week and hour of the day. For the day of the week, they need to tell us the number 3. How will they know that? Is Thursday day number 3, or is it Wednesday, or is it Tuesday? Does a week start on Sunday, or does it start on Monday, or does it start on Saturday? It depends on where in the world you are. Those are the kinds of things that are hard. Should the hour of the day be 18, 08, or should it be 8? People don't know when they do the ML.PREDICT and this becomes a huge problem for productionization. That feature creation is something that we did, it's not really a business thing and we cannot expect that when someone calls ML.PREDICT that they know the exact set of transformations that we did.

So what do, is whenever you're doing your ML query, in your SELECT clause, you pull out the raw data. That's what you're seeing on the right. You're doing SELECT duration, start_station_name, and start_date. Those three things are the raw data, and all the transformations you put into the transform clause. You’re basically saying select everything except the start date, so you're going to get duration and start station name. Then you're saying: get me the day of the week and get me the hour of the day. Then you train your model with that. The cool thing is that BigQuery ML knows about all these transformations. So when we do the ML.PREDICT, we only need to give it 2 inputs: start_station_name and start_date, and it tell us the duration. So that's why when you look at the ML.PREDICT, we parse in Kings Cross and

CURRENT_TIMESTAMP and it will give us a predictive duration. That 350, which is the actual duration, just gets carried along and we can compare the predicted against the actual if you ever want to.

Proprietary + Confidential

Transformation functions

```
TRANSFORM(* EXCEPT(start_date)
                  , IF(EXTRACT(dayofweek FROM start_date) BETWEEN 2 and 6, 'weekday', 'weekend') as dayofweek
                  , ML.BUCKETIZE(EXTRACT(HOUR FROM start_date), [5, 10, 17]) AS hourofday
        )
```

ML.BUCKETIZE
ML.EMBED
ML.FEATURE_CROSS

etc.

Google Cloud

So what kind of transformation functions exist? You can call any SQL function inside the TRANSFORM class. That's why we were able to do EXTRACT. But we can also do IF. If this thing is between 2 and 6, weekday and weekend, that's a pretty complex transformation. Now our model is trained with weekend versus weekday, but the prediction query remains exactly the same. They're still going to give us a start date. We can bucketize the hour, so whenever they give us an hour, we're going to break it into four buckets: morning, afternoon, evening, and night. And the end user doesn't need to know. They still call ML.PREDICT with a start date that transformations get done automatically. So you have a variety of transformation functions and you're seeing a few of them here: bucketize, embed, feature cross, etc. Go off and read about them to figure out what they do.

Proprietary + Confidential

The finer details ...

Supported regions

BigQuery ML supports the same regions as BigQuery

Quotas

You are limited to 1,000 `CREATE MODEL` queries per day per project

Pricing

BigQuery ML pricing is based on your usage pattern: flat-rate or on-demand

Google Cloud

So what are some of the finer details?

- First of all, BigQuery ML supports the same regions as BigQuery, so you can use it everywhere where you can use BigQuery.
- You're limited to 1,000 CREATE MODEL queries per day per project.
- The pricing is really based on your usage pattern. So if you are in a flat rate, you get a bunch of reservations and those same reservations get used to train the BigQuery ML model. So it's no different from any other queries. But if you're on-demand, BigQuery ML is unlike your typical query where you get charged for a table scan. You're going to read the data once and that's what you get charged for. But with machine learning, you don't read the data just once, you read the data multiple times to train with it. It's actually a multiplier, so you get charged a number multiplied by the size of your data. Look at your contract to figure out what that number is.

Proprietary + Confidential

Agenda

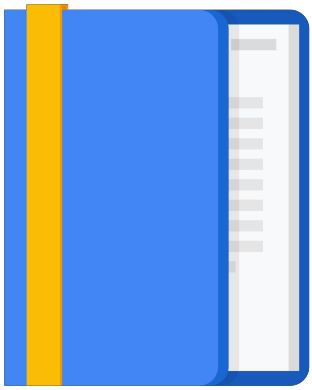
BigQuery GIS

Lab: Getting started with BigQuery GIS - New York City Bikes

BigQuery ML

Lab: Predict Taxi Fare with a BigQuery ML Forecasting Model

Summary



Google Cloud

You will now have the opportunity to predict a taxi fare with a BigQuery ML forecasting model.

Lab Intro

Predict Taxi Fare with a BigQuery ML Forecasting Model

In this lab, you will explore NYC taxi cab trips in a BigQuery Public Dataset and create an ML model inside of BigQuery to predict the fare of the cab ride given their model inputs.

Proprietary + Confidential

Lab objectives

- Use BigQuery to find public datasets
- Query and explore the public taxi cab dataset
- Create a training and evaluation dataset to be used for batch prediction
- Create a forecasting (linear regression) model in BigQuery ML
- Evaluate the performance of your ML model

Google Cloud

During the lab, you will learn how to:

- Use BigQuery to find public datasets,
- Query and explore the public taxi cab dataset,
- Create a training and evaluation dataset to be used for batch prediction,
- Create a forecasting (linear regression) model in BigQuery ML, and
- Evaluate the performance of your ML model.

Proprietary + Confidential

Agenda

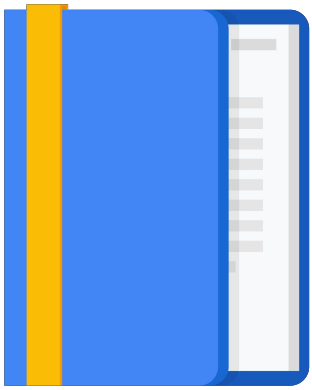
BigQuery GIS

Lab: Getting started with BigQuery GIS - New York City Bikes

BigQuery ML

Lab: Predict Taxi Fare with a BigQuery ML Forecasting Model

Summary



Google Cloud

That concludes the ‘BigQuery Extended Capabilities’ module. Let me summarize what you learned.

Proprietary + Confidential

Summary (1/2)

BigQuery GIS allows you to analyze geographic data in BigQuery

BigQuery is the first MPP data warehouse to support geospatial data types and functions

BigQuery GIS assumes that WKT points are defined in WGS-84

When you load BigQuery GIS data into BigQuery, you can specify a GEOGRAPHY column in the table's schema definition. BigQuery GIS can detect whether the data is in WKT or GeoJSON format

You can use a GEOGRAPHY column as a clustering column, but you cannot use a GEOGRAPHY column as a partitioning column

Google Cloud

- We looked at two things, BigQuery GIS and BigQuery ML. We discussed how BigQuery GIS allows you to analyze geographic data in BigQuery.
- We also discussed how BigQuery GIS was the first massively parallel processing data warehouse to support geographic data, pipes and functions.
- We looked at the WKT support, well-known text support, and the fact that when you define a point, it’s in WGS-84, which happens to be the same ellipsoid that GPS uses.
- And then we talked about the geography data types and how you could store the data in BigQuery as geographies, and internally, what BigQuery does is figure out the S2 cell coverage and that's what makes the querying very, very fast.

Proprietary + Confidential

Summary (2/2)

It's more performant to store the geography values in a BigQuery table

BigQuery uses standard SQL predicate functions

BigQuery ML enables users to create and execute ML models in BigQuery using standard SQL queries

View BigQuery ML model visualizations from within the BigQuery UI

Google Cloud

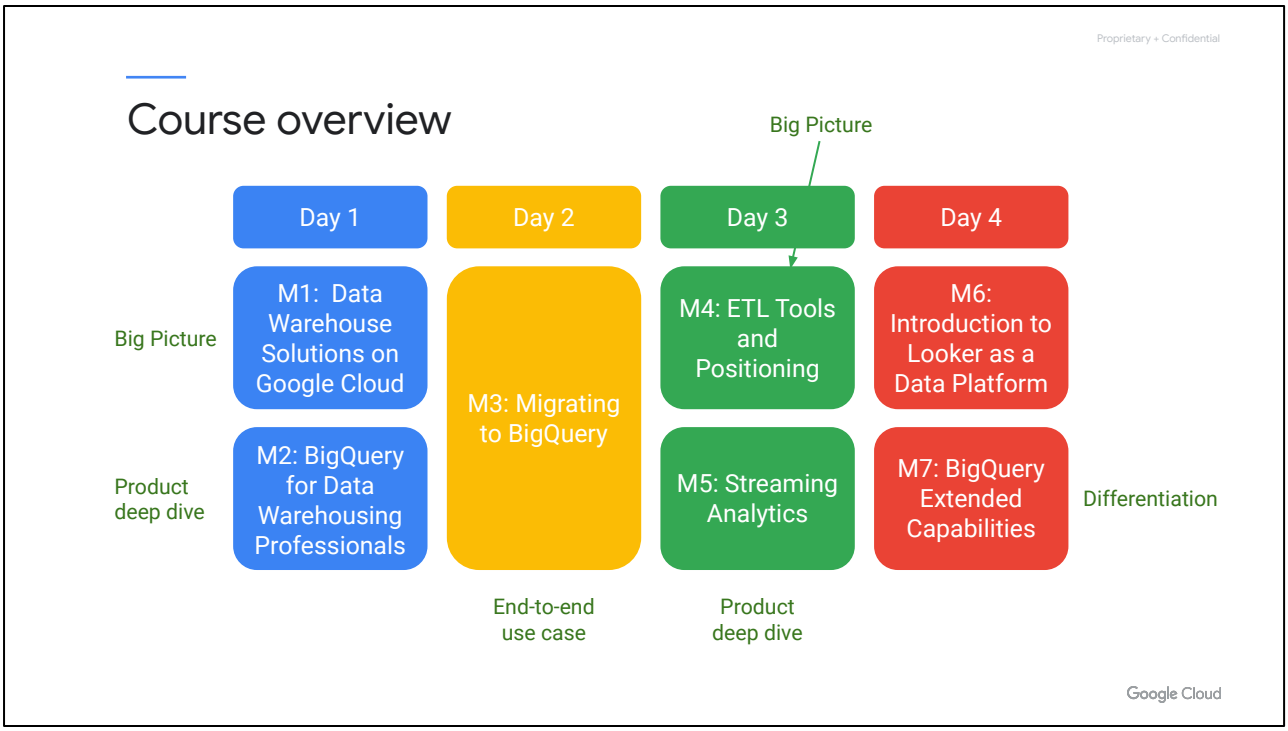
- We then talked about what you could use the geography columns for. You could use them as a WHERE clause, you could use them as a clustering column. But something that you cannot do is to use them as a partitioning column. Today in BigQuery you can partition based on time and you can partition based on integers, but not geographies.
- We then looked at the fact that when you want to store geographies you store them in the BigQuery table as a geography and that's what gives it the performance benefits.
- We looked at a variety of SQL predicate functions. These are the things that return booleans. We looked at things like contains, within, distance within, etc.
- Then we started looking at BigQuery ML. We said BigQuery ML is impressive because it allows users to create, train, and predict machine learning models using just SQL queries.
- Then we talked about how you can do the evaluation of BigQuery ML models in the BigQuery web console, and look at things like the confusion matrix.

So with that we've looked at what BigQuery GIS and BigQuery machine learning are able to do in terms of differentiations and what they provide on Google Cloud. It gives people a reason to migrate over to Google Cloud.

Data Warehousing for Partners: Data Warehouse Migration with BigQuery

Review

And this brings us to the end of our course on data warehousing for partners, where we looked at how to do data warehouse migration to BigQuery.



We looked at BigQuery for data warehousing professionals. We looked in detail about how to use BigQuery for doing data warehousing. We looked at how to migrate from an on premise data warehouse to BigQuery and how to do streaming analytics. We then explored Looker, Google Cloud's enterprise platform for data experiences. And we finally ended with some of the extended capabilities that BigQuery provides, such as GIS and machine learning. Thank you all very much for joining us on this course, good luck.